PHP and HTML Form Handling Documentation

PHP Objectives Covered

1. **Understand PHP Form Handling**: Process user input from HTML forms using the `$_POST` superglobal array.
2. **Implement Conditional Logic**: Use `isset()` function to check if form variables are set before processing.
3. **Work with Arrays**: Handle multiple checkbox selections and multiple list box selections as arrays.
4. **Apply Sorting Functions**: Sort array data using `sort()` function for organized output.
5. **Practice Output Formatting**: Display processed data using `echo` statements with proper formatting and HTML tags.
6. **Understand Form Submission**: Differentiate between GET and POST methods for form submission.

HTML Form Objectives Covered

1. **Create Various Input Types**: Text fields, checkboxes, radio buttons, list boxes, and textareas.
2. **Use Form Attributes**: Apply `method="post"` and `action="input_types.php"`.
3. **Apply Name and ID Attributes**: Map form inputs to PHP `$_POST` keys via `name` attributes.
4. **Implement Array Naming**: Use array notation (`name="hobby[]"`, `name="pantsSize[]"`) for multi-selection.
5. **Understand Form Submission and Reset**: Implement submit and reset buttons.

---

2. Visual Screenshots and Output Documentation

Screenshot 1: Initial Empty HTML Form (`input_types.html`)

The form renders all required input controls: single-line text inputs, single checkbox, multiple checkboxes, radio buttons, single select dropdown, multi-select list box, multi-line textarea, and action buttons.

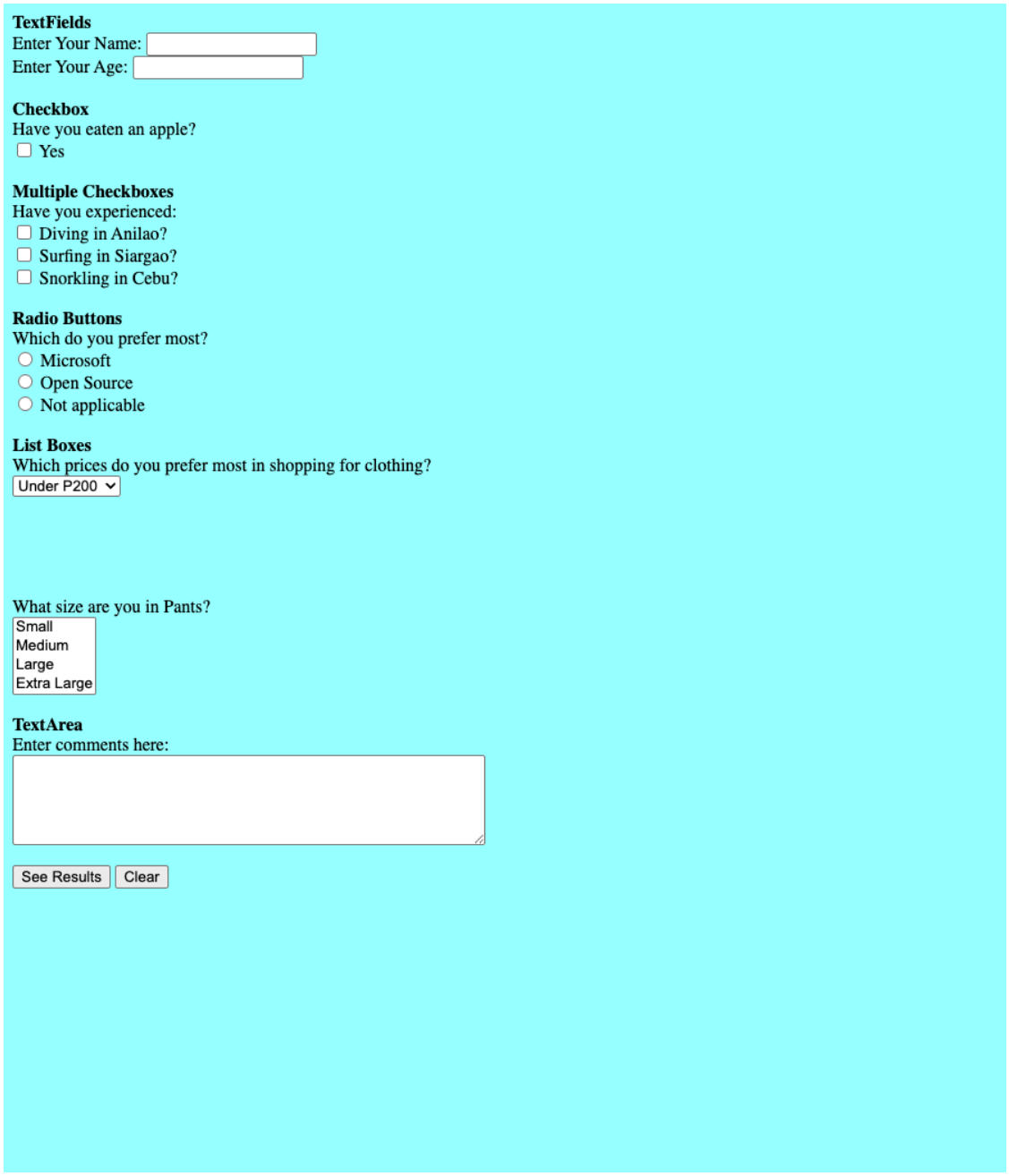


Figure 1: Initial Empty HTML Form

Screenshot 2: Form Populated with User Input

The user inputs sample values across all fields: - **Name**: `Juan Dela Cruz` - **Age**: `21` - **Single Checkbox**: Checked (`Okay`) - **Multiple Checkboxes**: All 3 checked (`Diving in Anilao`, `Surfing in Siargao`, `Snorkling in Cebu`) - **Radio Buttons**: `Open Source` - **List Box**: `P500-P1000` - **Multiple List Box**: `Medium`, `Large`, `Extra Large` - **TextArea**: Comments entered - **Submit Action**: Clicking `See Results`

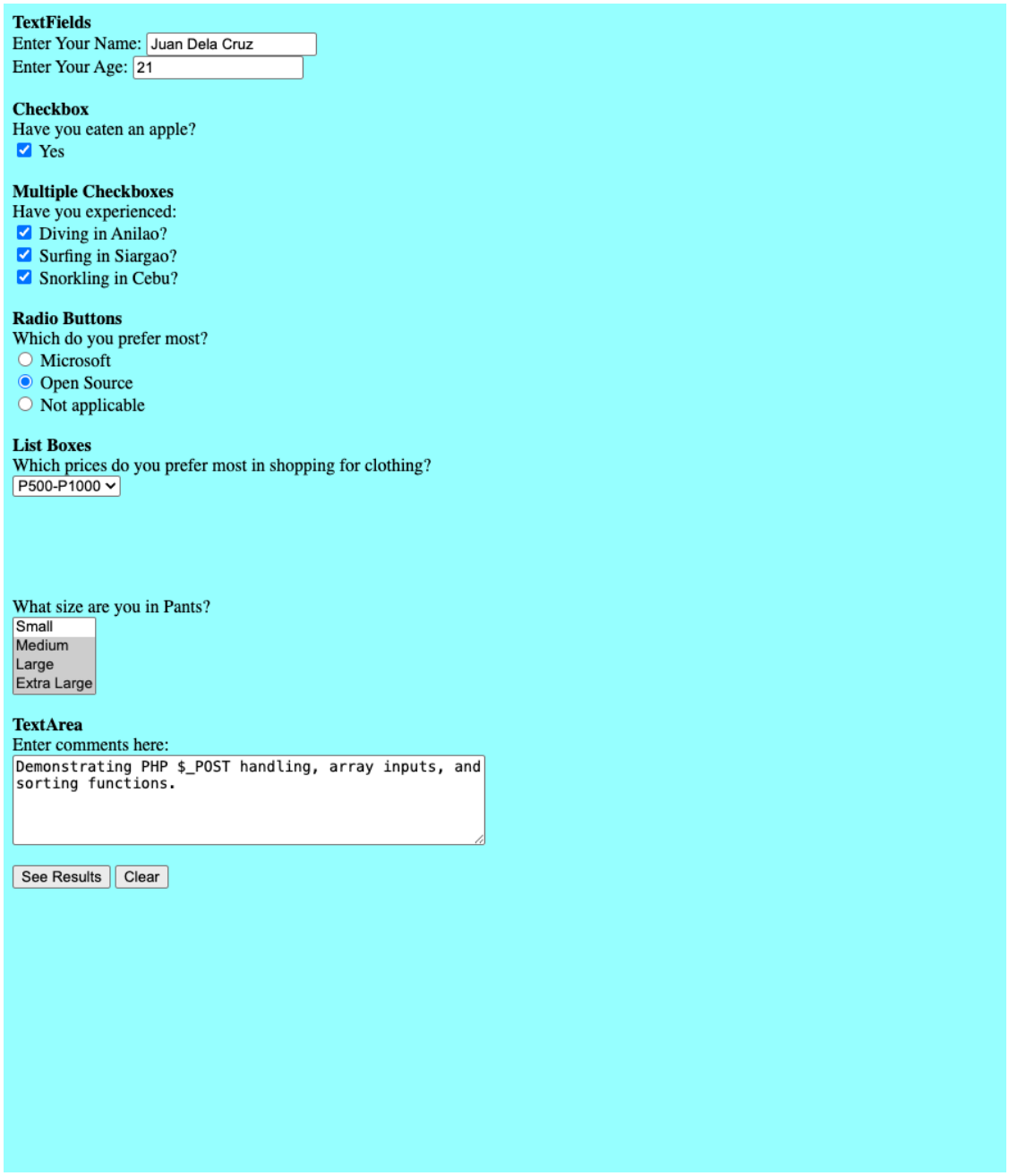


Figure 2: Form with User Input Values

Screenshot 3: Processed PHP Output (`input_types.php`) - Complete Submission

Upon submission, `input_types.php` receives the `$_POST` array, performs array sorting using `sort()`, and formats the output into ordered (`<ol>`) and unordered (`<ul>`) lists.

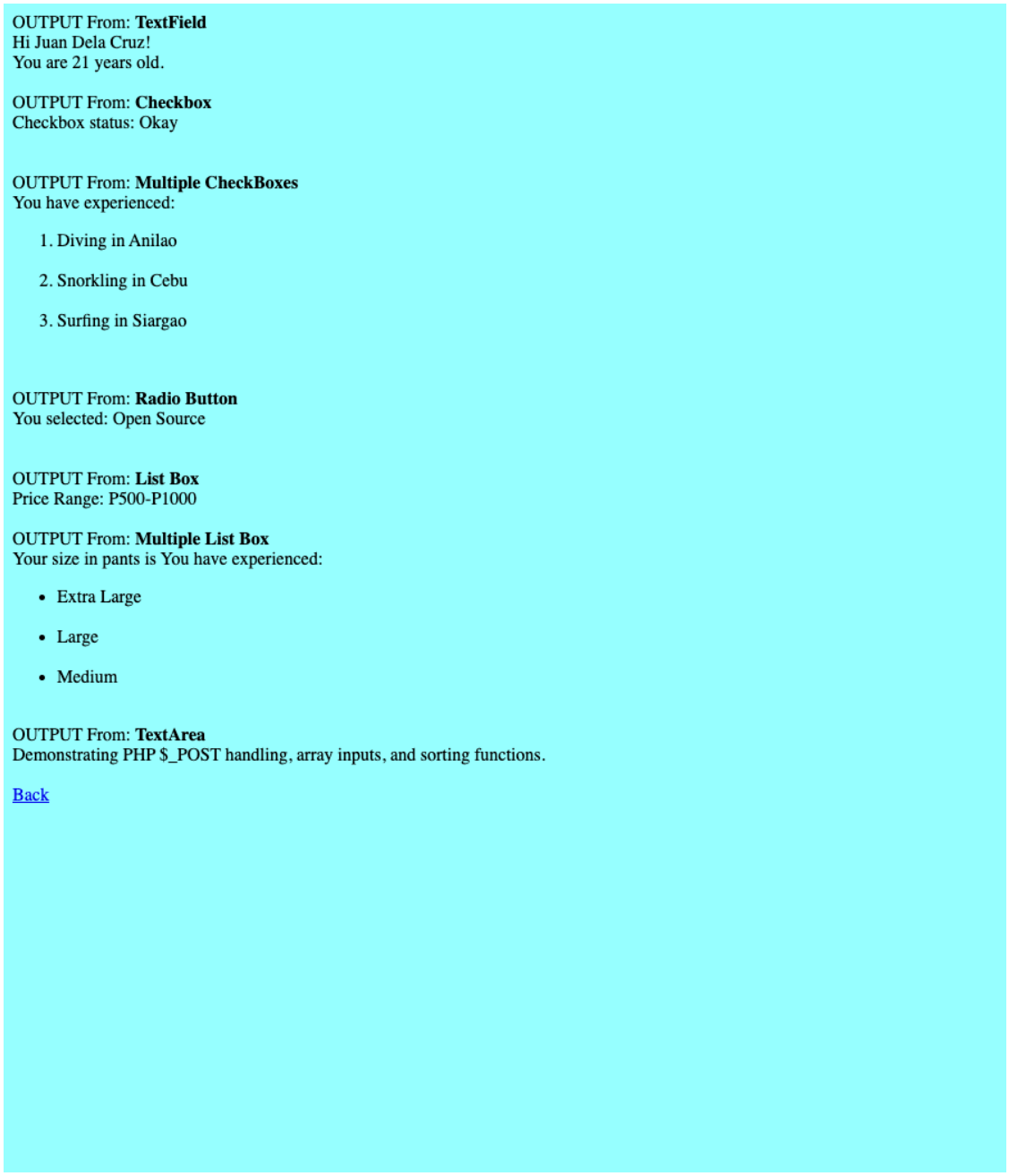


Figure 3: PHP Form Processing Output (Complete Submission)

Screenshot 4: Edge Case & Conditional Logic Output (`isset()` Analysis)

When the single checkbox is left unchecked, the browser does not send the key `choice` in the POST request. The script evaluates `isset($_POST['choice'])`.

OUTPUT From: **TextField**
Hi Maria Santos!
You are 19 years old.

OUTPUT From: **Checkbox**
Checkbox status:
Warning: Undefined array key "choice" in **/Users/keithalanspeirs/Documents/Test/input_types.php** on line **17**

OUTPUT From: **Multiple CheckBoxes**
You have experienced:

1. Diving in Anilao
2. Snorkling in Cebu

OUTPUT From: **Radio Button**
You selected: Microsoft

OUTPUT From: **List Box**
Price Range: Under P200

OUTPUT From: **Multiple List Box**
Your size in pants is You have experienced:

- Extra Large
- Large
- Small

OUTPUT From: **TextArea**
Testing conditional logic when checkbox is unchecked and array sorting with multiple sizes.

Back

Figure 4: PHP Output Showing Unchecked State & Conditional Handling

---

3. Technical Analysis and Detailed Explanations

A. HTML Form Elements Breakdown

HTML Element	Syntax in `input_types.html`	Purpose & Behavior
Text Fields	`<input type="text" name="pangalan"><input type="text" name="edad" id="age">`	Captures single-line strings. Accessed in PHP via `$_POST['pangalan']` and `$_POST['edad']`.
Single Checkbox	`<input type="checkbox" name="choice" value="Okay">`	Binary toggle. Transmits `choice=Okay` only when checked.
Multiple Checkboxes	`<input type="checkbox" name="hobby[]" value="...">`	Array notation `hobby[]` bundles all selected values into a PHP indexed array.
Radio Buttons	`<input type="radio" name="q1" value="Open Source">`	Mutual exclusion among inputs sharing `name="q1"`. Only selected value is sent.
Single List Box	`<select name="price"><option>...</option></select>`	Dropdown menu providing single value under `$_POST['price']`.
Multiple List Box	`<select name="pantsSize[]" multiple>`	Allows multi-item selection. Bundled into array `$_POST['pantsSize']`.
TextArea	`<textarea name="comments" cols="50" rows="5">`	Multi-line text field accessible via `$_POST['comments']`.
Action & Reset	`<input type="submit" value="See Results"><input type="reset" value="Clear">`	`submit` triggers HTTP POST payload; `reset` clears client form fields.

B. PHP Form Processing & Superglobal `$_POST`

When the form is submitted via `POST`, PHP creates an associative array `$_POST`:

```php
$name = $_POST['pangalan'];
echo "OUTPUT From: <b> TextField </b> <br>";
echo "Hi $name!<br>";
echo "You are {$_POST['edad']} years old. <br><br>";
```

C. Conditional Logic with `isset()`

Checkboxes and unselected controls are omitted from the HTTP payload. The `isset()` function verifies if a variable exists:

```php
if (isset($_POST['q1'])) {
    echo "{$_POST['q1']} <br><br>";
}
```

D. Array Handling and Sorting (`sort()`)

For multi-selection inputs (`hobby[]` and `pantsSize[]`), PHP constructs indexed arrays. The `sort()` function sorts the array elements alphabetically:

```php
if (isset($_POST['hobby'])) {
    $choi = $_POST['hobby'];
    $n = count($choi);
    $i = 0;
    echo "You have experienced: <br><ol>";
    while ($i < $n) {
        sort($choi);
        echo "<li>{$choi[$i]}</li><br>";
        $i++;
    }
    echo "</ol>";
}
```

Sorting Demonstration: - Unsorted Input: `["Surfing in Siargao", "Diving in Anilao", "Snorkling in Cebu"]` - After `sort()`: 1. *Diving in Anilao* 2. *Snorkling in Cebu* 3. *Surfing in Siargao*

E. GET vs. POST Methods

Feature	GET Method	POST Method
Data Transmission	Appended to URL query string	Sent inside HTTP request body
Visibility	Visible in browser address bar	Hidden from address bar

Feature	GET Method	POST Method
Data Capacity	Limited (~2KB)	Substantially larger limit
Security	Not suitable for passwords/forms	Preferred for form handling
Idempotency	Read/Search queries	Data submission and state change

---

4. Code Review and Best Practice Recommendations

1. Fix for Unchecked Single Checkbox

In `input_types.php` lines 14–17:

```
// Issue: Line 17 accesses $_POST['choice'] even when unset
if (isset($_POST['choice']))
    echo "{$_POST['choice']} </b> <br><br>";
else
    echo "{$_POST['choice']} </b> <br><br>"; // Generates PHP 8 Warning
```

Recommended Fix:

```
if (isset($_POST['choice'])) {
    echo htmlspecialchars($_POST['choice']) . "</b><br><br>";
} else {
    echo "No / Not Selected</b><br><br>";
}
```

2. Optimization for `sort()` Loop

In `input_types.php`, `sort()` is repeatedly called inside the `while` loop. Calling it once before loop iteration is more efficient:

```
if (isset($_POST['hobby'])) {
    $choi = $_POST['hobby'];
    sort($choi); // Sort once
    echo "You have experienced:<br><ol>";
    foreach ($choi as $hobby) {
        echo "<li>" . htmlspecialchars($hobby) . "</li><br>";
    }
    echo "</ol>";
}
```